

Foundations of Spatial AI

Lecture 02 — 3D Representations

Detailed lecture notes

Notes based on the slides by L. Di Giammarino and E. Giacomini
Sapienza University of Rome

Slides dated September 28, 2026

Abstract

These notes follow the slide deck of Lecture 02 (75 slides) in the order in which the slides present the material. Every section header carries the range of slides it covers. Formal statements are typeset in boxes (**Definition**, **Proposition**, **Algorithm**, ...), and every result or non-obvious idea is followed by a blue *Intuition* box. All figures are original TikZ/pgfplots redrawings, so the document is self-contained. The last slide of the file (slide 75, a backup slide placed after the acknowledgments) is collected in Appendix ??.

Contents

1	Representations for 3D (<i>slides 2–4</i>)	3
2	Notation (<i>slide 5</i>)	3
3	Manifolds (<i>slide 6</i>)	4
4	Defining a Surface (<i>slide 7</i>)	5
5	Meanwhile in 3D-land... (<i>slides 8–9</i>)	5
6	Depth Maps (<i>slides 10–14</i>)	6
6.1	Depth maps (<i>slides 10–12</i>)	6
6.2	Depth Maps and Normal Maps (<i>slide 13</i>)	6
6.3	Depth Maps — Representing the Visible (<i>slide 14</i>)	6
7	Point Clouds (<i>slides 15–21</i>)	7
7.1	Point Clouds (<i>slides 15–18</i>)	7
7.2	Oriented Point Clouds (<i>slide 19</i>)	8
7.3	No explicit connectivity (<i>slides 20–21</i>)	8
8	Meshes (<i>slides 22–34</i>)	9
8.1	Meshes: vertices and faces (<i>slides 22–25</i>)	9
8.2	Triangular Meshes (<i>slides 26–31</i>)	10
8.3	Triangular Meshes — Geometry vs Connectivity (<i>slides 32–34</i>)	12
9	Parametric Surfaces (<i>slides 35–41</i>)	13
9.1	Parametric surfaces in 2D (<i>slide 35</i>)	13
9.2	Parametric surfaces (<i>slides 36–40</i>)	13
9.3	Parametric Surfaces — Pros and Cons (<i>slide 41</i>)	15

10 Surface Representations (<i>slide 42</i>)	16
11 Volume Representations (<i>slides 43–46</i>)	16
11.1 Voxelized 3D (<i>slides 44–46</i>)	16
12 Implicit Surfaces (<i>slides 47–62</i>)	18
12.1 Implicit surfaces (<i>slide 47</i>)	18
12.2 Generate vs Test (<i>slide 48</i>)	18
12.3 Implicit Surfaces: Examples (<i>slide 49</i>)	19
12.4 When a Field Defines a Surface (<i>slide 50</i>)	19
12.5 Normals for Free (<i>slide 51</i>)	20
12.6 Boolean Operations (<i>slide 52</i>)	21
12.7 Many Functions, One Surface (<i>slide 53</i>)	21
12.8 Signed Distance Function (<i>slides 54–58</i>)	21
12.9 Storing a Field on a Grid (<i>slide 59</i>)	23
12.10 Storing a Field in a Network (<i>slide 60</i>)	24
12.11 From Implicit Surface to Mesh (<i>slide 61</i>)	24
12.12 Implicit Surfaces: Summary (<i>slide 62</i>)	25
13 Boundary vs Volumetric Representations (<i>slide 63</i>)	25
14 Density Fields (<i>slides 64–65</i>)	25
14.1 Density fields (<i>slide 64</i>)	25
14.2 Primitive-based Density Fields (<i>slide 65</i>)	25
15 So? (<i>slide 66</i>)	26
16 And many more... (<i>slides 67–71</i>)	26
16.1 Octrees (<i>slide 68</i>)	26
16.2 Layered Depth Images (<i>slide 69</i>)	26
16.3 Multi-plane Images (<i>slide 70</i>)	27
16.4 CAD Models and CSG Trees (<i>slide 71</i>)	27
17 Summary (<i>slides 72–74</i>)	27

1 Representations for 3D (*slides 2–4*)

The deck opens with the same bunny (and a knot) shown as a triangle mesh, a point cloud, a smooth surface and a voxel grid: the *same shape*, four very different data structures.

Life is nice in 2D-land... (*slide 3*)

In 2D, every camera (an old box camera, a compact camera, a phone) produces the same object: a regular grid of pixels. That single representation is shared across *sensors*, *tools* (Photoshop) and *learning frameworks* (PyTorch, TensorFlow). Nobody asks “which image representation is best”.

Meanwhile in 3D land... (*slide 4*)

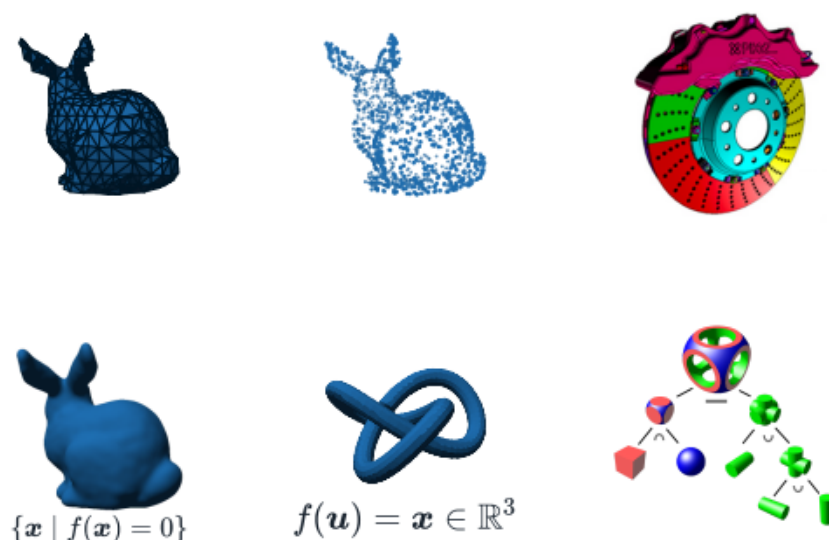


Figure 1: The same shape represented in four different ways

In 3D there is no such consensus. The slide lists: triangle meshes, point clouds, CAD models, surfaces given as a zero level set $\{\mathbf{x} \mid f(\mathbf{x}) = 0\}$, surfaces given by a parametrization $f(\mathbf{u}) = \mathbf{x} \in \mathbb{R}^3$, CSG trees, “and many more”.

Intuition

The 2D grid is universal because image sensors *measure* on a grid and every algorithm can assume that grid. In 3D, different tasks pull in different directions: a LiDAR returns unordered points, a designer edits smooth curves, a physics engine needs inside/outside tests, a neural network prefers fixed-size arrays. Each 3D representation is a different compromise. The rest of the lecture is a tour of these compromises.

2 Notation (*slide 5*)

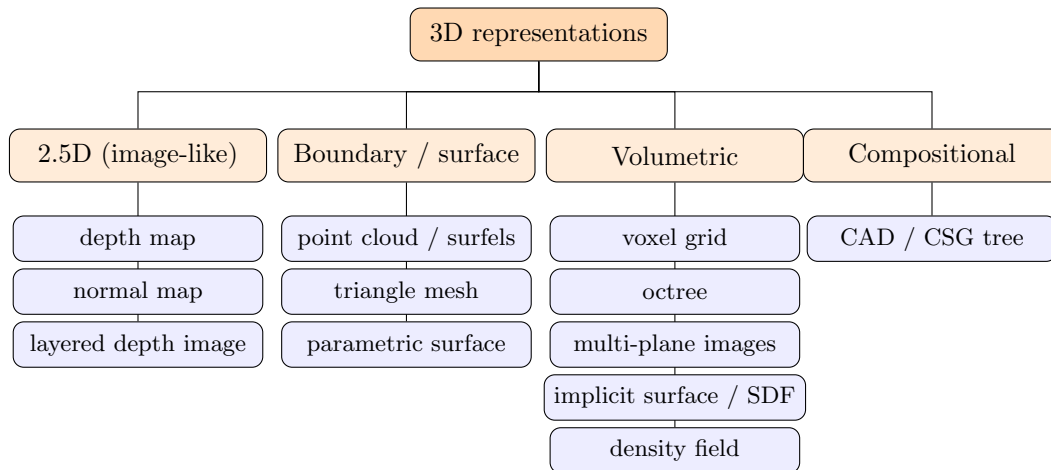


Figure 2: A map of the representations met in the lecture (our grouping, following slides 43 and 63: points, meshes and parametric surfaces are *boundary* representations; voxels and implicit fields are *volumetric*).

Definition 2.1 (Notation). A point in 3D space is written in inhomogeneous coordinates as

$$\mathbf{x} = \begin{pmatrix} x \\ y \\ z \end{pmatrix} \in \mathbb{R}^3.$$

A point on the image plane carries the subscript s : $\mathbf{x}_s \in \mathbb{R}^2$. Discrete maps are *indexed* rather than evaluated, and brackets denote indexing:

$$D[\mathbf{x}_s] \in \mathbb{R}^+ \quad (\text{depth}), \quad N[\mathbf{x}_s] \in \mathbb{S}^2 \quad (\text{unit normal}), \quad V[i, j, k] \in [0, 1] \quad (\text{occupancy}).$$

Remark 2.2. The subscript denotes the *coordinate frame*, not the dimension of the space. Both $\mathbf{x}$ and $\mathbf{x}_s$ denote points.

Intuition

Parentheses mean “compute” ($f(\mathbf{x})$ runs a formula or a network); brackets mean “look up” ($D[\mathbf{x}_s]$ reads a pixel). The letter s is a hint that $\mathbf{x}_s$ lives on the screen; it is a name tag, not an exponent.

3 Manifolds (*slide 6*)

Definition 3.1 (Manifold). An n -dimensional *manifold* is a topological space that locally resembles Euclidean space: every point has a neighborhood homeomorphic to an open subset of $\mathbb{R}^n$; n is its dimension.

Lines and circles are 1-manifolds. Two-dimensional manifolds are called *surfaces*: the plane, the sphere, the torus. Each coloured piece of the shape on the slide is a *chart*: it flattens a part of the shape onto $\mathbb{R}^n$.

Intuition

An ant walking on a sphere sees a flat plane around itself: that local flatness is what “manifold” means. The sphere as a whole is *not* a plane, which is why one needs several overlapping

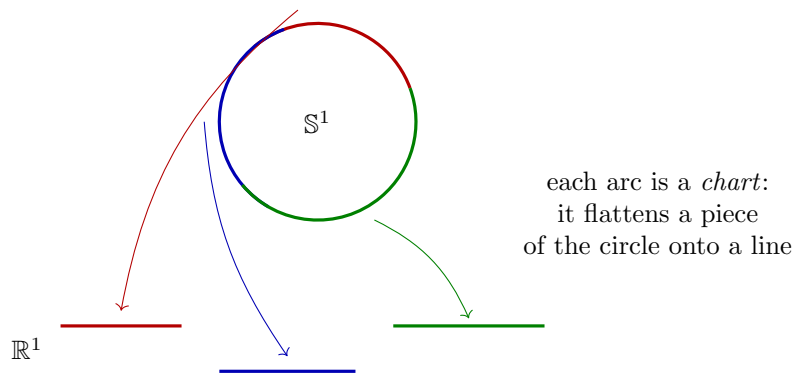


Figure 3: The circle is a 1-manifold. Three overlapping charts, each a homeomorphism from an arc to an interval of $\mathbb{R}$, cover it. (Slide 6 shows the same idea for a circle and a sphere.)

charts (an atlas) to describe it, exactly as one needs several flat maps to cover the Earth without cutting it.

4 Defining a Surface (*slide 7*)

Definition 4.1 (Parametric and implicit forms). A surface can be specified in two ways.

- **Parametric form:** a function maps a 2D domain into space, $f(\mathbf{u}) = \mathbf{x} \in \mathbb{R}^3$, $\mathbf{u} \in \mathcal{M}$.
- **Implicit form:** the surface is the zero level set of a scalar field, $\mathcal{S} = \{\mathbf{x} \mid f(\mathbf{x}) = 0\}$.

In the parametric form the domain point $\mathbf{u}$ is *not* a point in space: it belongs to a 2D manifold $\mathcal{M}$, for instance the sphere $\mathbb{S}^2$.

Remark 4.2. The symbol f denotes both functions; the argument decides which is meant: $f(\mathbf{u})$ returns a surface point, $f(\mathbf{x})$ returns a scalar.

Intuition

Parametric = “a recipe that *produces* points” (feed in a domain point, get a 3D point out).
Implicit = “a rule that *judges* points” (feed in a 3D point, get a number whose sign says on which side of the surface it lies). Slide 48 will turn this into the mantra *parametric generates, implicit tests*.

5 Meanwhile in 3D-land... (*slides 8–9*)

“But what is the *best* representation?” The slide’s answer: *it depends*, on the task: acquisition vs editing vs animation vs prediction vs processing vs optimization, and more.

The goals of the lecture (slide 9):

- understand representations *computationally* (what do they look like in code?);
- understand their limitations and benefits;
- understand conversions across representations.

Intuition

There is no free lunch: a representation that makes one operation trivial tends to make another one painful. Reading each representation below, ask three questions: how do I *store* it, what is *easy* to do with it, what is *hard*? The lecture proceeds from representations tied to sensors (depth maps, point clouds), through surfaces (meshes, parametric surfaces), to

volumes (voxels, implicit fields, density fields).

6 Depth Maps (*slides 10–14*)

6.1 Depth maps (*slides 10–12*)

Depth is a *common modality across sensors* (structured-light and time-of-flight cameras such as the Kinect and industrial cameras shown on slide 10); depth is treated in detail in Lectures 04–05.

Definition 6.1 (Depth map). A depth map is an image $D[\mathbf{x}_s] \in \mathbb{R}^+$ that records how far the scene point seen by each pixel $\mathbf{x}_s$ is from the camera.

Because it is just an image, *standard image-processing tools can be used* on it (slide 12): filtering, resizing, convolutional networks.

6.2 Depth Maps and Normal Maps (*slide 13*)

The same idea extends to other surface properties, e.g. surface normals:

Definition 6.2 (Normal map). $N[\mathbf{x}_s] \in \mathbb{S}^2$ stores, for each pixel, the unit normal of the visible surface.

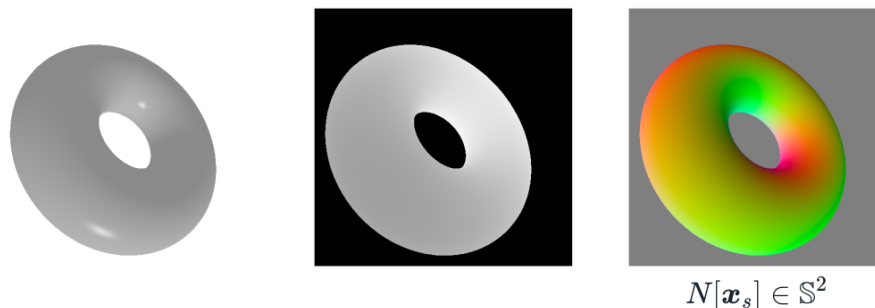


Figure 4: Depth and normal maps of a scene. The depth map is a grayscale image where brightness encodes distance; the normal map is a color image where RGB encodes the XYZ components of the unit normal.

6.3 Depth Maps — Representing the Visible (*slide 14*)

Remark 6.3. A depth map does not capture the *full* 3D structure: it is a *2.5D* representation, i.e. properties associated with image pixels. The slide asks: can we develop more *complete* 3D representations?

Intuition

A depth map is like a relief map seen from one viewpoint: it tells the distance to whatever is *in front*, and nothing about what is behind it. Two things follow. First, cheap and fast: it is an ordinary image. Second, incomplete: to model a whole object you need several views or a representation that lives in 3D itself, which motivates point clouds next.

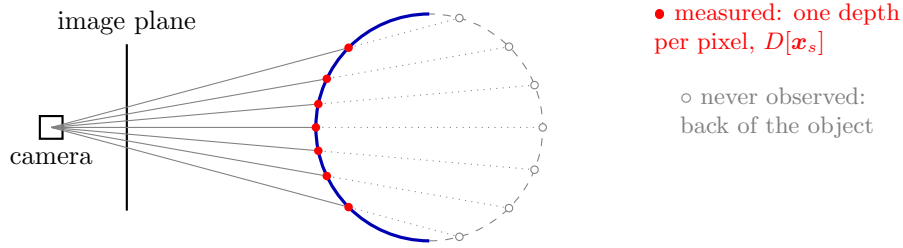


Figure 5: Side view of a depth camera. Each pixel ray returns the *first* surface hit (red); the far side of the object (dashed) and anything hidden behind the visible surface is absent from D .

7 Point Clouds (*slides 15–21*)

7.1 Point Clouds (*slides 15–18*)

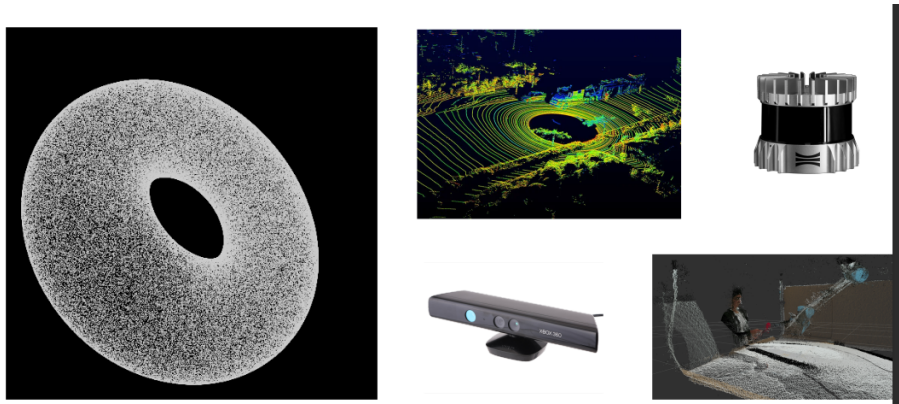


Figure 6: Some point clouds of scenes. Each point is a 3D position, and can be drawn as a small sphere.

Point clouds are often acquired in an image-like manner from sensors (LiDAR, depth cameras) but it can be useful to *forget* this: point clouds from multiple views can be merged into a single point cloud, which helps unify approaches independently of the sensing modality.

Definition 7.1 (Point cloud). A point cloud is an *unordered set* of points $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\} \subset \mathbb{R}^3$, usually stored as an $N \times 3$ array. Reordering the rows by any permutation σ gives $\{\mathbf{x}_{\sigma(1)}, \dots, \mathbf{x}_{\sigma(N)}\}$, the *same* point cloud.

Unlike an image, the row order carries no meaning, so processing or generation methods must be *permutation invariant*; a fully connected layer applied to the flattened array does not work.

Example 7.2 (Why a fully connected layer fails). Let $X = \{\mathbf{x}_1 = (1, 0, 0), \mathbf{x}_2 = (0, 1, 0)\}$. Flattened, the two orderings give $(1, 0, 0, 0, 1, 0)$ and $(0, 1, 0, 1, 0, 0)$, two different vectors, so a generic linear layer returns two different outputs for one geometric object. A *symmetric* operation such as taking the per-coordinate maximum over the points returns $(1, 1, 0)$ in both orders.

Intuition

Think of a bag of marbles: shaking it does not change what it contains. An operator must treat the input as a bag, not as a list. Symmetric functions (sum, mean, max over points) have this property, and network designs for point clouds are built from them.

7.2 Oriented Point Clouds (*slide 19*)

Definition 7.3 (Oriented point cloud, surfel). An oriented point cloud stores a normal with each point: $\{(x_1, n_1), \dots, (x_N, n_N)\}$. Points can then be treated as small flat disks rather than tiny spheres, which helps rendering under lighting. When a radius is also stored the primitives are called *surfels*.

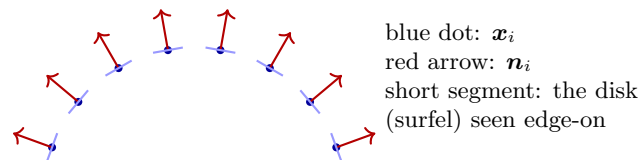


Figure 7: An oriented point cloud sampled from a curve (2D cross-section): each sample carries a position and a normal, and can be drawn as a small disk perpendicular to the normal.

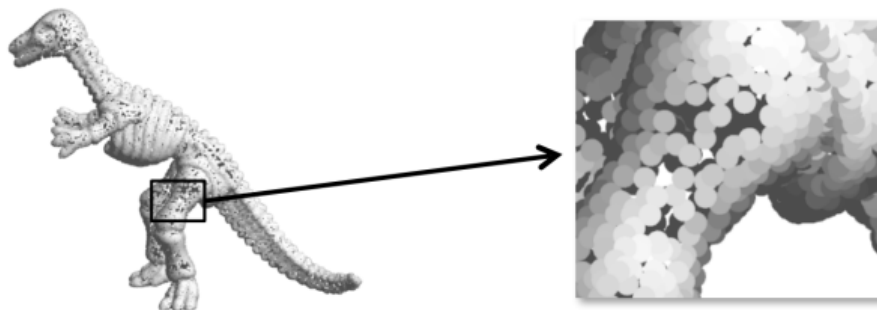


Figure 8: A surfel-based reconstruction of a dinosaur. Each surfel is a small disk with a position, a normal and a radius.

Intuition

A bare point has no facing direction, so lighting is ambiguous. Attaching a normal turns each point into a tiny mirror; adding a radius makes the mirror cover a patch of surface, so neighbours overlap and the gaps between samples close up.

7.3 No explicit connectivity (*slides 20–21*)

A point cloud has no explicit *connectivity* information. Two point sets that look nearly identical may come from a closed circle and from a circle with a small gap. One can always sample more points to resolve the gap, but this is super inefficient; it is more efficient to add *edges*, because connectivity does not scale like density (which grows with an area).

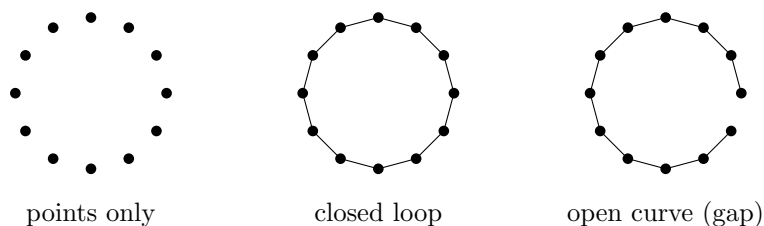


Figure 9: The same 12 points support two different shapes: only the edges (connectivity) tell them apart.

Example 7.4 (Density vs. connectivity). To sample a unit sphere with spacing h one needs roughly $4\pi/h^2$ points: about 1257 for $h = 0.1$ but about 5027 for $h = 0.05$. Halving the spacing multiplies the number of points by 4, while adding one edge costs one pair of integers.

Intuition

Density is a $2D$ bill (points per unit area), connectivity is a $1D$ one (a few edges along features). To say “these two neighbouring points belong to the same continuous piece” it is far cheaper to draw a line than to fill the gap with hundreds of extra samples. This is exactly what meshes do.

8 Meshes (*slides 22–34*)

8.1 Meshes: vertices and faces (*slides 22–25*)

Meshes are *piecewise linear approximations* of the underlying surface; more elements allow a better approximation (slide 22 shows a torus with increasing resolution).

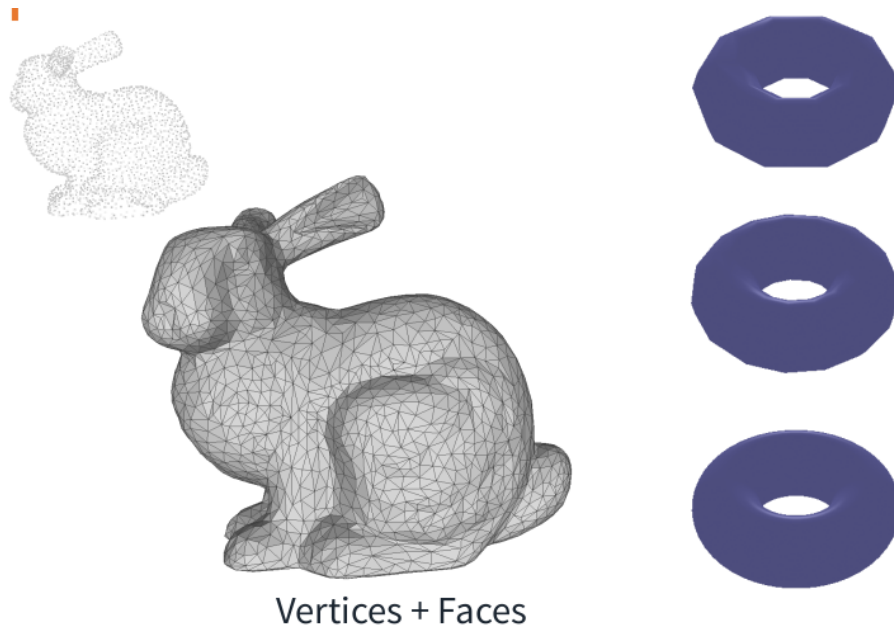


Figure 10: A torus approximated by a mesh of increasing resolution and the difference between a mesh and a pointcloud of a rabbit. The mesh is a piecewise linear approximation of the underlying surface.

Definition 8.1 (Mesh). A mesh is a pair (V, F) : V is an $N \times 3$ array of vertex positions (the *geometry*) and F is a list of M faces (the *topology*), each face being a list of indices of the vertices that make it up. Consecutive indices in a face are joined by an edge, so *ordering matters* in face indices.

Arbitrary polygons can also be *non-planar* (four points in 3D need not lie in a plane), so one restricts to triangles: three points not on a line always define a plane.

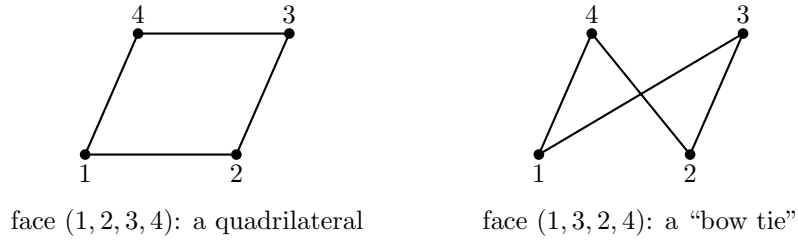


Figure 11: Same four vertices, two index orders, two different shapes: consecutive indices are connected by an edge.

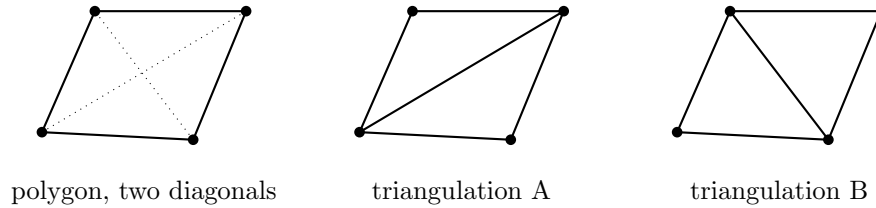


Figure 12: A quad can be split into triangles in two ways. If the four points are not coplanar, A and B are two genuinely different surfaces, while each triangle on its own is always flat.

8.2 Triangular Meshes (*slides 26–31*)

Definition 8.2 (Triangular mesh). A triangular mesh is a mesh whose faces are all triangles: $V \in \mathbb{R}^{N \times 3}$ and $F \in \{0, \dots, N-1\}^{M \times 3}$, each row of F listing the indices of the *three* vertices of a face.

Example 8.3 (A tetrahedron in code).

```
V = np.array([[0,0,0],[1,0,0],[0,1,0],[0,0,1]], dtype=float) # 4 x 3
F = np.array([[0,2,1],[0,1,3],[0,3,2],[1,2,3]])           # 4 x 3, ints
```

Here $N = 4$ vertices, $M = 4$ faces and $E = 6$ edges.

A sanity check on sizes.

Proposition 8.4 (Face count of a closed genus-0 triangle mesh). *Let a closed, sphere-like (genus 0) triangular mesh have V vertices, E edges and F faces. Every face has 3 edges and every edge is shared by 2 faces, so $E = 3F/2$. With Euler’s formula $V - E + F = 2$ this gives*

$$F = 2V - 4.$$

Proof. Substitute $E = 3F/2$ into $V - E + F = 2$: $V - \frac{3}{2}F + F = 2$, i.e. $V - \frac{1}{2}F = 2$, hence $F = 2V - 4$. $\square$

Intuition

You get about *twice as many triangles as vertices*. Checks: tetrahedron $V = 4 \Rightarrow F = 4$ ✓; octahedron $V = 6 \Rightarrow F = 8$ ✓; a scan with $V = 10,000$ vertices has $F = 19,996$ faces and $E = 3F/2 = 29,994$ edges. Useful to guess memory footprints or to spot a broken mesh.

Orientation. For a triangle, *any order gives the same triangle*, but the order determines which side is “front”. The normal is $\mathbf{n} = (\mathbf{b} - \mathbf{a}) \times (\mathbf{c} - \mathbf{a})$.

Proposition 8.5 (Orientation and index order). *A cyclic shift $(a, b, c) \rightarrow (b, c, a)$ leaves $\mathbf{n}$ unchanged; swapping two indices, e.g. $(a, b, c) \rightarrow (c, b, a)$, flips its sign.*

Proof. Expanding, $(\mathbf{b} - \mathbf{a}) \times (\mathbf{c} - \mathbf{a}) = \mathbf{b} \times \mathbf{c} - \mathbf{b} \times \mathbf{a} - \mathbf{a} \times \mathbf{c} + \mathbf{a} \times \mathbf{a} = \mathbf{a} \times \mathbf{b} + \mathbf{b} \times \mathbf{c} + \mathbf{c} \times \mathbf{a}$. This expression is unchanged by cyclic relabelling $\mathbf{a} \rightarrow \mathbf{b} \rightarrow \mathbf{c} \rightarrow \mathbf{a}$ and changes sign under the swap of any two symbols, since $\times$ is antisymmetric. $\square$



(a, b, c) : counter-clockwise, $\mathbf{n}$ toward the viewer (c, b, a) : clockwise, $\mathbf{n}$ away from the viewer

Figure 13: Index order defines the winding direction and hence the side the normal points to. Swapping two indices reverses it.

Example 8.6 (Checking the tetrahedron). For face $(0, 2, 1)$ of the example above, $\mathbf{a} = (0, 0, 0)$, $\mathbf{b} = (0, 1, 0)$, $\mathbf{c} = (1, 0, 0)$: $\mathbf{n} = (0, 1, 0) \times (1, 0, 0) = (0, 0, -1)$, pointing down, i.e. *outward* for the face lying in $z = 0$ (the tetrahedron sits above it). Face $(1, 2, 3)$ gives $(-1, 1, 0) \times (-1, 0, 1) = (1, 1, 1)$, also outward. All four faces are consistently oriented.

Intuition

The “classic bug”: if some faces are listed clockwise and others counter-clockwise, the renderer treats some triangles as back-facing and they render black or vanish. Rule of thumb: use one winding convention for the whole mesh and check with $\mathbf{n}$ pointing away from the interior.

Manifold problems. Even with triangles one can still get *non-manifold* meshes. The slides identify two cases: an edge shared by more than two faces (so restrict each edge to at most two faces; one can often get away without enforcing this) and a *disconnected surface around a vertex*, which is critical for mesh-processing applications and less so for rendering.



an edge seen end-on, shared by 3 faces two fans touching at a single vertex

Figure 14: Two kinds of non-manifold configuration.

Ray-triangle intersection and texturing.

Proposition 8.7 (Barycentric form and the ray-triangle solve). *Any point of the triangle with vertices $\mathbf{a}, \mathbf{b}, \mathbf{c}$ is $\alpha\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c}$ with $\alpha + \beta + \gamma = 1$ and all three weights ≥ 0 . Setting this*

equal to the ray $\mathbf{o} + t\mathbf{d}$ gives 3 linear equations in the 3 unknowns (t, β, γ) , i.e. a 3×3 solve per triangle:

$$\beta(\mathbf{b} - \mathbf{a}) + \gamma(\mathbf{c} - \mathbf{a}) - t\mathbf{d} = \mathbf{o} - \mathbf{a}, \quad \alpha = 1 - \beta - \gamma.$$

The ray hits the triangle iff $\beta \geq 0$, $\gamma \geq 0$, $\beta + \gamma \leq 1$ (and $t > 0$).

Proof. Substitute $\alpha = 1 - \beta - \gamma$ in $\alpha\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c} = \mathbf{o} + t\mathbf{d}$ and collect terms. $\square$

Example 8.8. Triangle $\mathbf{a} = (0, 0, 0)$, $\mathbf{b} = (1, 0, 0)$, $\mathbf{c} = (0, 1, 0)$; ray $\mathbf{o} = (0.25, 0.25, 1)$, $\mathbf{d} = (0, 0, -1)$. The system reads $\beta(1, 0, 0) + \gamma(0, 1, 0) - t(0, 0, -1) = (0.25, 0.25, 1)$, so $\beta = \gamma = 0.25$, $t = 1$, $\alpha = 0.5$. All weights are ≥ 0 : hit at $\mathbf{o} + \mathbf{d} = (0.25, 0.25, 0)$.

Texturing is easy: each vertex also stores a coordinate into a 2D image, and inside a triangle the same weights interpolate those coordinates.

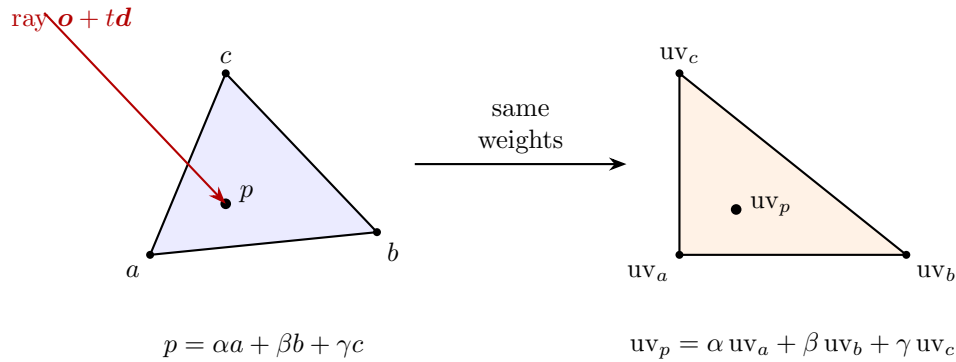


Figure 15: Left: the hit point in barycentric coordinates $(\alpha, \beta, \gamma) = (0.5, 0.25, 0.25)$. Right: the same weights interpolate the 2D texture coordinates stored at the vertices.

Intuition

Barycentric weights are “how much of each corner” a point is made of; a point near a has α close to 1. Intersecting a ray with a triangle is therefore a tiny linear-algebra problem, which is why GPUs can do billions of them, and the *same* weights carry any per-vertex quantity (texture coordinates, colours, normals) to the interior.

8.3 Triangular Meshes — Geometry vs Connectivity (slides 32–34)

The two arrays of a mesh behave very differently: V is `float`, differentiable, and gradient descent (GD) is happy; F is `int`, discrete, and GD is not happy. Modifying vertex positions can edit the shape; other changes are non-trivial (i.e. changing F).

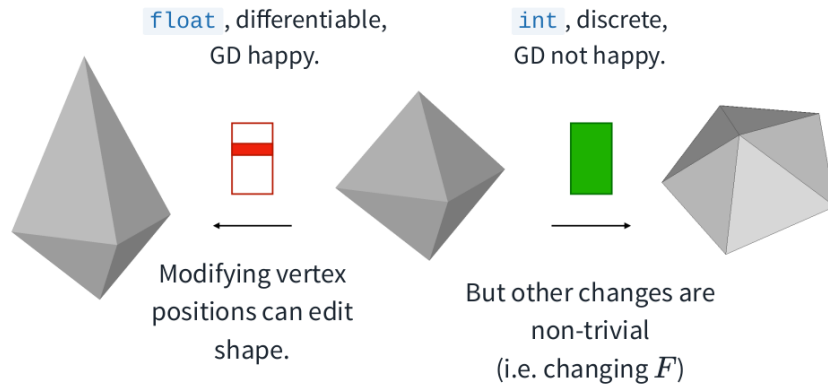


Figure 16: Editing the geometry V is easy, editing the connectivity F is hard.

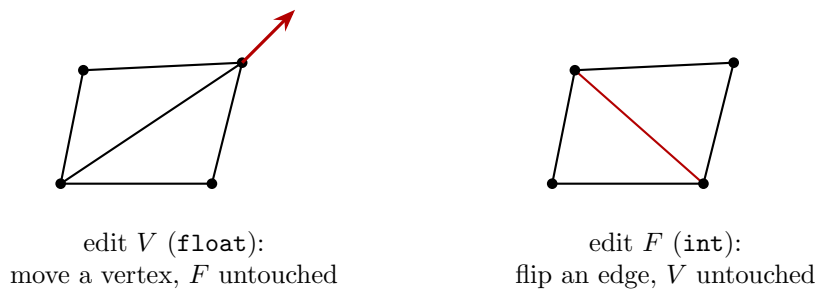


Figure 17: Geometry edits are small continuous changes of real numbers; connectivity edits are combinatorial jumps.

Same counts, different meshes (*slide 33*)

Going from one mesh to another is non-trivial even when both have 6 vertices and 8 faces (the octahedron is one such mesh). It is even more challenging if the topology changes, e.g. torus to sphere. In general, designing methods that can predict *arbitrary* connectivity is challenging (e.g. image to mesh).

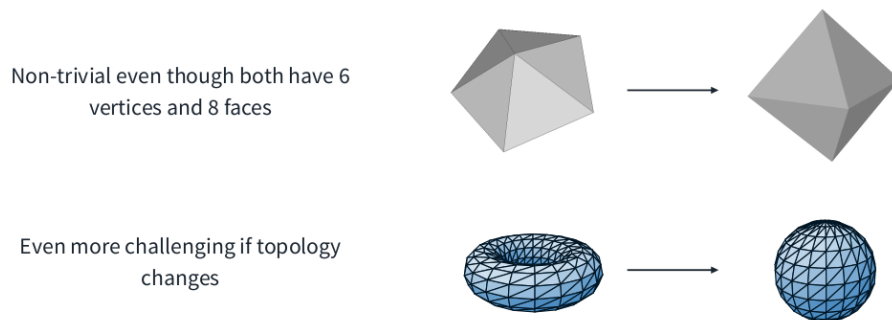


Figure 18: Two meshes with the same number of vertices and faces, but different connectivity.

Continuous deformation (*slide 34*)

The first change (same counts, different connectivity) is possible under continuous deformation of the shape; the torus-to-sphere change is impossible under continuous deformation (a hole cannot be closed without tearing). Yet *both* edits require similar operations for meshes: a new F and an

updated V . The slide asks: is there an alternate representation where the former (the continuous one) is easier?

Intuition

A mesh mixes a smooth part (V) with a rigid combinatorial part (F). Even a tiny, continuous change of the shape may demand a jump in F , and a network cannot easily output “an integer array of unknown length”. The question hanging in the air motivates representations where shape changes are always changes of continuous numbers: parametric and, above all, implicit ones.

9 Parametric Surfaces (*slides 35–41*)

9.1 Parametric surfaces in 2D (*slide 35*)

The domain is a circle $\mathbb{S}^1$; think of it as an angle θ . A function $f(\mathbf{u}) = \mathbf{x} \in \mathbb{R}^2$, $\mathbf{u} \in \mathbb{S}^1$ sends each angle to a point of a curve: f_1 to a pentagon, f_2 to a square.

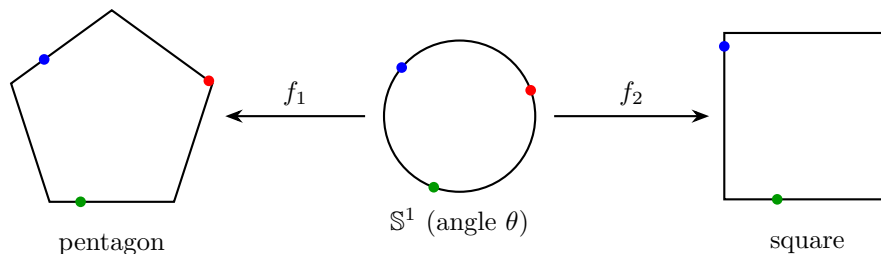


Figure 19: Two different maps f_1, f_2 from the same circle domain. Same colour = same domain point θ .

9.2 Parametric surfaces (*slides 36–40*)

Definition 9.1 (Parametric surface). A continuous function $f : \mathcal{M} \rightarrow \mathbb{R}^3$, $f(\mathbf{u}) = \mathbf{x}$, over a 2D manifold $\mathcal{M}$ parametrizes a surface. The *connectivity/topology* is constrained to be the same as $\mathcal{M}$. This disentangles discretization (vertices/faces) from shape representation. In code:

```
def f(u):
    """Input: u: N-dimensional vector from a 2D manifold
       Output: x: 3D point"""
```

f can be a fixed analytic function, a parametrized family, or even a neural network $f_\theta : \mathbb{S}^2 \rightarrow \mathbb{R}^3$.

Example 9.2 (Analytic examples). The saddle $f((u, v)) = (u, v, v^2 - u^2)$; and the trefoil-knot tube of slide 38:

```
def trefoil(u, v):
    # r is just a float scale factor
    x = r * np.sin(3*u) / (2 + np.cos(v))
    y = r * (np.sin(u) + 2*np.sin(2*u)) / (2 + np.cos(v + np.pi*2/3))
    z = r/2 * (np.cos(u) - 2*np.cos(2*u)) * (2 + np.cos(v)) * (2 + np.cos(v + np.pi*2/3)) / 4
```

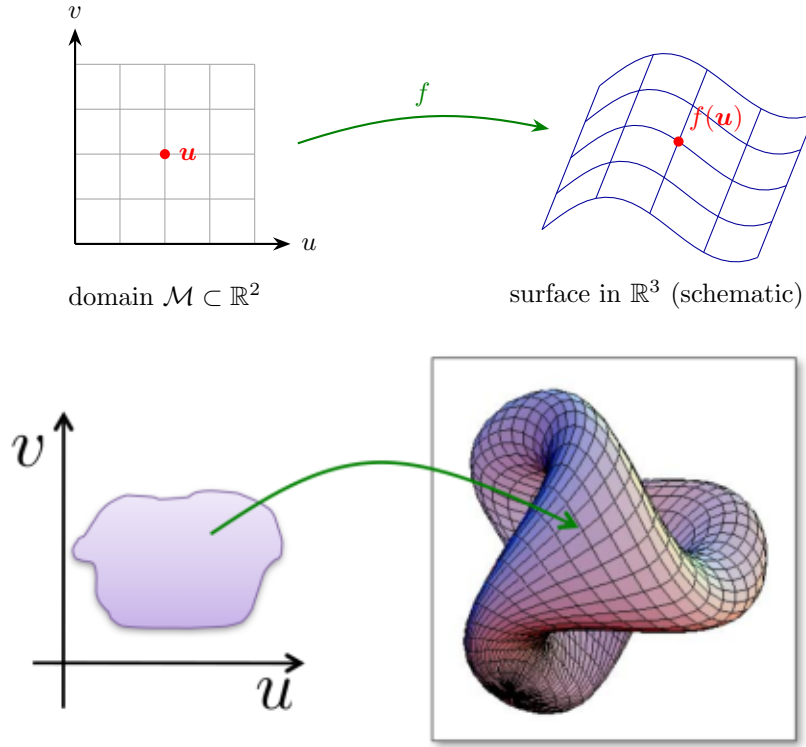


Figure 20: A parametrization warps a flat domain into a surface: grid lines of (u, v) become curves on the surface, and neighbouring parameters stay neighbouring.

Definition 9.3 (Bézier surface). With $(N + 1)(M + 1)$ control points $\mathbf{k}_{i,j} \in \mathbb{R}^3$,

$$f((u, v)) = \sum_{i=0}^N \sum_{j=0}^M B_i^N(u) B_j^M(v) \mathbf{k}_{i,j}, \quad B_i^N(u) = \binom{N}{i} u^i (1 - u)^{N-i}.$$

Here N and M are polynomial *degrees*, not counts. The control points determine a smooth surface in 3D; Bézier patches are commonly used for designing meshes.

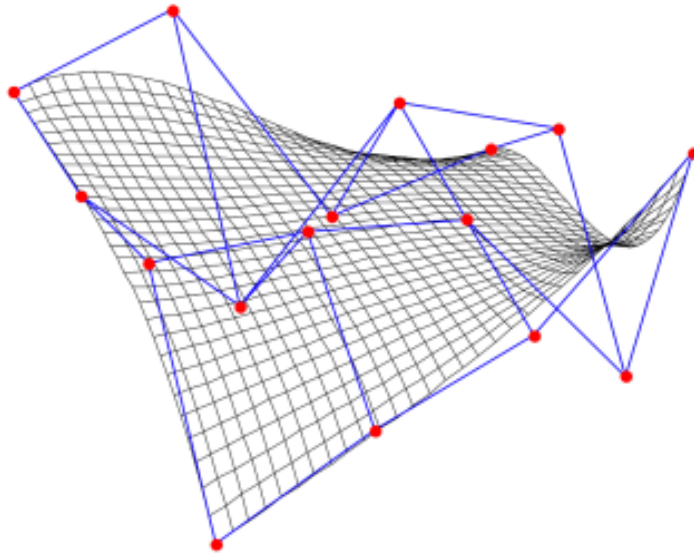


Figure 21: A Bézier surface

Example 9.4 (The bilinear patch, $N = M = 1$). $B_0^1(u) = 1 - u$, $B_1^1(u) = u$, so $f = (1 - u)(1 - v)\mathbf{k}_{00} + (1 - u)v\mathbf{k}_{01} + u(1 - v)\mathbf{k}_{10} + uv\mathbf{k}_{11}$. With $\mathbf{k}_{00} = (0, 0, 0)$, $\mathbf{k}_{10} = (1, 0, 0)$, $\mathbf{k}_{01} = (0, 1, 0)$, $\mathbf{k}_{11} = (1, 1, 1)$: $f(0.5, 0.5) = \frac{1}{4}[(1, 0, 0) + (0, 1, 0) + (1, 1, 1)] = (0.5, 0.5, 0.25)$.

Intuition

Control points act like a coarse cage of pins: move a pin and the smooth sheet follows. The surface is a weighted average of the pins, with weights (Bernstein polynomials) that sum to one, so it stays inside the convex hull of the cage. That is why the same $(N + 1)(M + 1)$ numbers describe an entire smooth surface no matter how finely you later sample it.

9.3 Parametric Surfaces — Pros and Cons (*slide 41*)

- **Sampling is easy:** pick $\mathbf{u}$, evaluate f .
- **Computation in the domain:** neighbours, texture coordinates and so on are just nearby $\mathbf{u}$ values.
- **“Is $\mathbf{q}$ inside?” is hard:** you would have to search for the $\mathbf{u}$ whose $f(\mathbf{u})$ is closest to $\mathbf{q}$, an optimization problem.
- **Rendering is hard:** a ray hits the surface where $\mathbf{o} + t\mathbf{d} = f(\mathbf{u})$, a nonlinear system in $(t, \mathbf{u})$. In practice you convert to a mesh first.

Intuition

Generating is a *forward* computation (plug in, read off); testing or intersecting needs the *inverse* of f , which is generally not available in closed form. The price of the parametrization is also its topology: the surface has whatever topology $\mathcal{M}$ has (a sphere domain can only give sphere-like shapes).

10 Surface Representations (*slide 42*)

Points, meshes and parametric surfaces all describe the “skin” of an object, and none of them can turn a torus into a sphere continuously. So what can we do?

Intuition

All three fix the topology in their data (the connectivity F , the domain $\mathcal{M}$, or the absence of connectivity). To let shapes change topology smoothly, one needs a representation that assigns a value to *every point of space* rather than tracking the skin.

11 Volume Representations (*slides 43–46*)

Volume representations give a value to every point in space.

11.1 Voxelized 3D (*slides 44–46*)

Definition 11.1 (Voxel grid). A voxel grid is a discretized representation of the space demarcated via cuboid endpoints $(x_{\min}, y_{\min}, z_{\min})$ and $(x_{\max}, y_{\max}, z_{\max})$: a $W \times H \times D$ array $V[i, j, k] \in [0, 1]$ storing the occupancy (or probability of occupancy) of each cell.

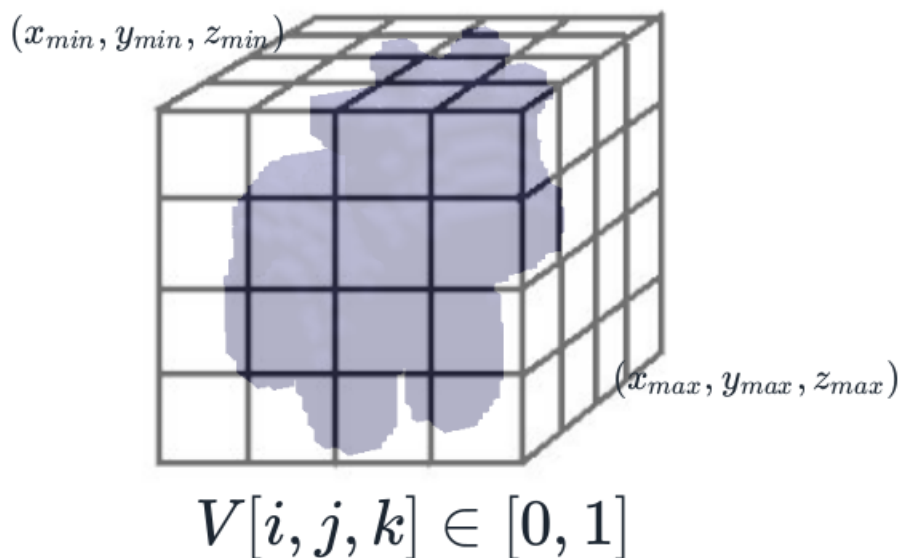


Figure 22: A $4 \times 4 \times 4$ voxel grid inside a cuboid; shaded cells are occupied.

Coding tip: disentangle *grid coordinates* from the *spatial extent in the world*. One common convention: cell size $h = (x_{\max} - x_{\min})/W$ along x , and cell i has its centre at $x_{\min} + (i + \frac{1}{2})h$; the same for y, z . For $x_{\min} = -1.5$, $x_{\max} = 1.5$, $W = 128$: $h \approx 0.0234$, and the point $x = 0.3$ falls in cell $i = \lfloor (0.3 + 1.5)/h \rfloor = 76$.

Because the representation is “image-like”, *convolutions* can be used for processing and generation. It is a generic way to represent shapes *across topologies*: the sphere, the torus and the cow of slide 45 all use the same array shape, and topology is just which cells are switched on. A fixed-size output is exactly what networks like.

Resolution is expensive (*slide 46*)

It is computationally challenging to scale resolution ($32^3, 64^3, 128^3, 256^3$ on the slide). Worse, the surface passes through only about n^2 of the n^3 cells, so almost all the memory describes empty space or solid interior.

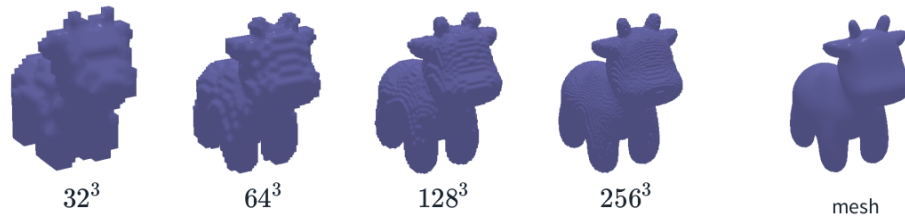


Figure 23: A representation of how resolution affects surface detail. The surface passes through only about n^2 of the n^3 cells, so almost all the memory describes empty space or solid interior.

Example 11.2 (Memory of a voxel grid).

n	cells n^3	surface cells $\approx n^2$	fraction $\approx 1/n$
32	32 768	1 024	3%
64	262 144	4 096	1.6%
128	2 097 152	16 384	0.8%
256	16 777 216	65 536	0.4%

At one byte per cell, 256^3 is 16 MB (64 MB with 32-bit floats) for a single object.

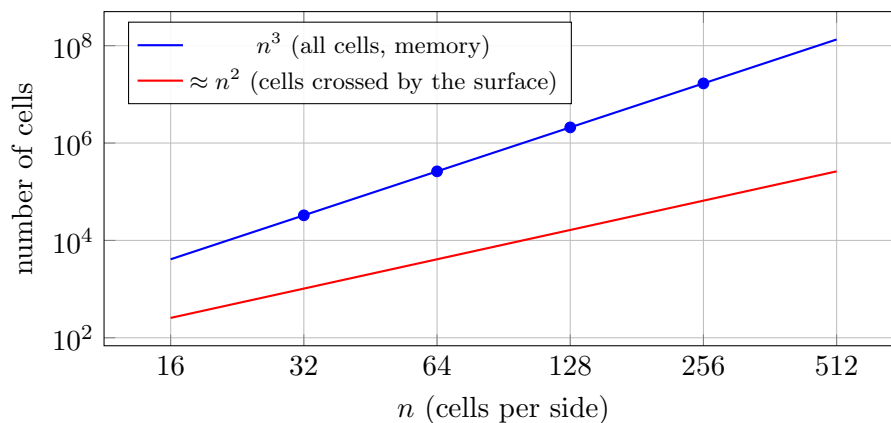


Figure 24: Doubling the resolution multiplies memory by 8 but the useful (surface) part only by 4.

Intuition

A voxel grid is a bookkeeping device: it pays the same for air as for the object. Doubling the resolution costs $8\times$ more memory but only $4\times$ more surface. The benefit is simplicity: any topology, fixed size, convolutions apply. The cost is that fine detail is very expensive, which motivates adaptive versions (octrees, slide 68) and continuous fields (next section).

12 Implicit Surfaces (*slides 47–62*)

12.1 Implicit surfaces (*slide 47*)

Definition 12.1 (Implicit surface). A surface is the *zero level set* of a scalar field $f : \mathbb{R}^3 \rightarrow \mathbb{R}$:

$$\mathcal{S} = \{\mathbf{x} \in \mathbb{R}^3 \mid f(\mathbf{x}) = 0\}.$$

The sign tells the side: $f(\mathbf{x}) < 0$ inside, $f(\mathbf{x}) > 0$ outside (one convention, used for the rest of the lecture).

12.2 Generate vs Test (*slide 48*)

The unit circle, written both ways:

	Parametric	Implicit
Definition	$g(u) = (\cos u, \sin u)$, $u \in [0, 2\pi)$	$f(x, y) = x^2 + y^2 - 1$
Give me 100 points on it	easy: evaluate g at 100 values of u	hard: you have to search for zeros
Is $(0.5, 0.5)$ inside?	hard: no direct test	easy: $f = -0.5 < 0$, so inside

Parametric representations *generate* points; implicit representations *test* points.

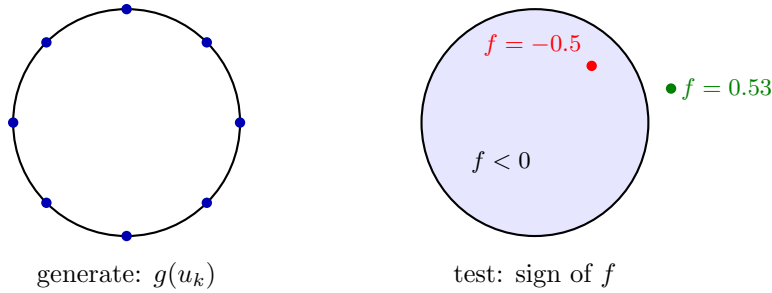


Figure 25: Left: the parametric circle produces points. Right: the implicit circle classifies any point by the sign of f .

Intuition

Parametric $\approx$ a machine that *prints* surface points; implicit $\approx$ a *bouncer* that checks tickets. Printing 100 points is trivial with the printer and awkward with the bouncer (you have to hunt for the zeros); checking a single point is trivial with the bouncer and awkward with the printer (you must invert g).

12.3 Implicit Surfaces: Examples (*slide 49*)

Simple functions can give concise representations of complex shapes:

sphere: $f(x, y, z) = x^2 + y^2 + z^2 - 1$,

torus: $f(x, y, z) = (x^2 + y^2 + z^2 + R^2 - a^2)^2 - 4R^2(x^2 + y^2)$,

quartic (blob with a hole): $f(x, y, z) = x^4 + y^4 + z^4 + 2x^2y^2 + 2x^2z^2 + 2y^2z^2 + 8xyz - 10x^2 - 10y^2 - 10z^2 + 20$.

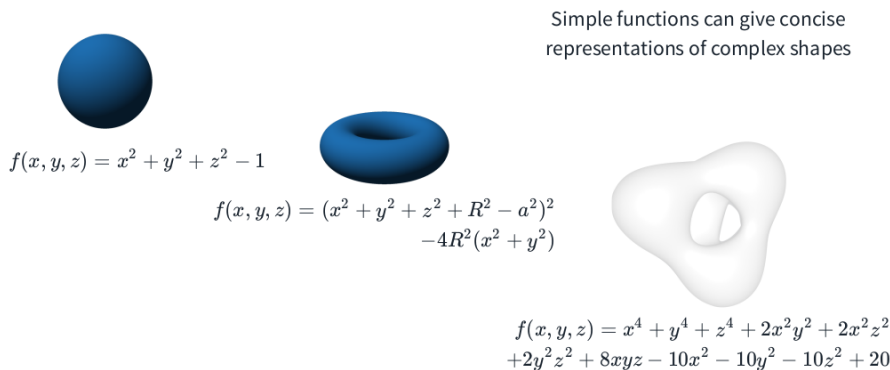


Figure 26: Implicit surfaces: sphere, torus, and a quartic with a hole. The last one is a single polynomial of degree 4, yet it has a hole in it.

12.4 When a Field Defines a Surface (slide 50)

Not every field defines a surface. Four test cases (with $r = \|\mathbf{x}\|$):

$f(\mathbf{x})$	zero set	a surface?
$\ \mathbf{x}\ - 1$	unit sphere	yes
$\ \mathbf{x}\ ^2 + 1$	empty	no, there is nothing there
$\ \mathbf{x}\ ^2$	the origin	no, a single point
$(\ \mathbf{x}\ - 1)^2$	unit sphere	no: $f \geq 0$ everywhere, so no inside, and $\nabla f = 0$ on it

Proposition 12.2 (When f defines a surface). f must change sign across $\mathcal{S}$, and $\nabla f \neq 0$ on $\mathcal{S}$. (If f is continuously differentiable and $\nabla f \neq 0$ at every zero of f , the implicit function theorem shows that $\mathcal{S}$ is locally the graph of a smooth function, i.e. a smooth surface.)

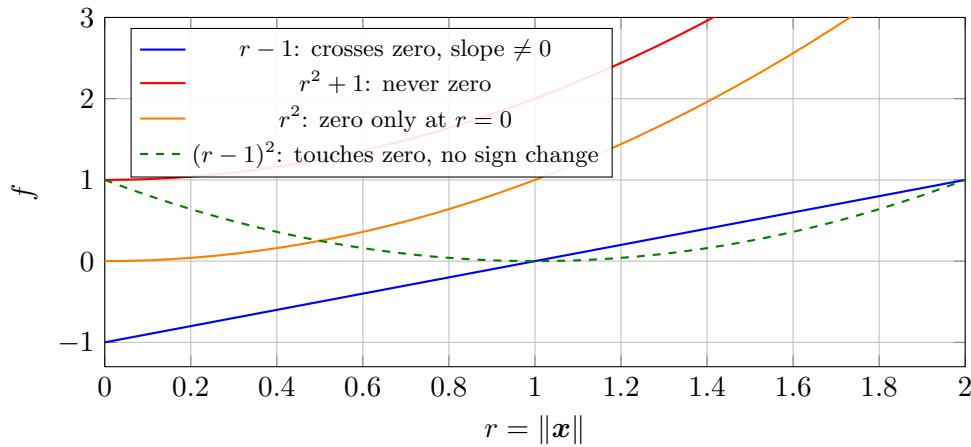


Figure 27: Radial profiles of the four candidate fields. Only the first one changes sign at the unit sphere with a non-zero slope.

Intuition

A surface is where the field *crosses* zero, like the shoreline where altitude crosses sea level. A field that only touches zero (the last row) gives a double root: no “inside”, no well-defined normal. A field that never reaches zero gives nothing.

12.5 Normals for Free (slide 51)

Lemma 12.3 (Normal of an implicit surface). *The gradient is perpendicular to the level sets, so on the surface*

$$\mathbf{n}(\mathbf{x}) = \frac{\nabla f(\mathbf{x})}{\|\nabla f(\mathbf{x})\|}.$$

It points toward increasing f , which by our convention is outward.

Proof. Let $\gamma(t)$ be any curve lying in $\mathcal{S}$. Then $f(\gamma(t)) = 0$ for all t ; differentiating, $\nabla f(\gamma(t)) \cdot \gamma'(t) = 0$, so ∇f is orthogonal to every tangent direction of the surface. $\square$

For the sphere, $\nabla f = 2\mathbf{x}$ and $\mathbf{n} = \mathbf{x}/\|\mathbf{x}\|$. With automatic differentiation the normals of *any* differentiable f (an MLP, say) come for free:

```

x = pts.clone().requires_grad_(True)      # (N, 3) points
v = f(x)                                  # any differentiable f, e.g. an MLP
g, = torch.autograd.grad(v.sum(), x)
n = g / g.norm(dim=-1, keepdim=True)     # (N, 3) unit normals

```

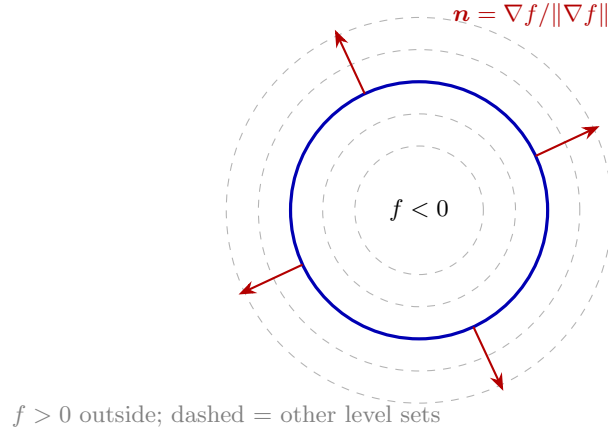


Figure 28: The gradient of f is orthogonal to the level sets and points toward larger f , i.e. out of the shape.

Intuition

Picture f as terrain height and the surface as the zero contour line. The gradient points straight uphill, perpendicular to any contour line: that is the normal. No separate normal array has to be stored or kept consistent, unlike the per-face orientation of a mesh.

12.6 Boolean Operations (*slide 52*)

Proposition 12.4 (Booleans via min and max). *With $f < 0$ inside,*

$$f_{A \cup B} = \min(f_A, f_B), \quad f_{A \cap B} = \max(f_A, f_B), \quad f_{A \setminus B} = \max(f_A, -f_B).$$

Proof. $x \in A \cup B \iff f_A(x) < 0$ or $f_B(x) < 0 \iff \min(f_A, f_B) < 0$. Likewise $x \in A \cap B \iff f_A < 0$ and $f_B < 0 \iff \max(f_A, f_B) < 0$. For $A \setminus B = A \cap B^c$, the complement of B is described (up to its boundary) by $-f_B < 0$, so $\max(f_A, -f_B) < 0$. $\square$

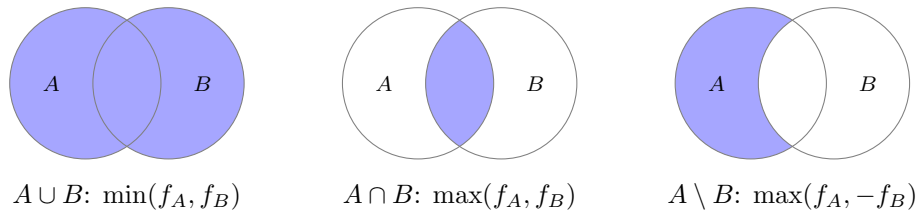


Figure 29: 2D versions of the three Boolean operations (blue = result).

Intuition

“Inside the union” means “inside at least one”: the most negative of the two values decides, hence min. “Inside the intersection” needs both, so the *least* negative value decides: max. Subtraction is intersection with the outside of B , and flipping the sign of f_B swaps inside and outside. The *sign* of the result is always correct; the magnitude near sharp creases is not

guaranteed to remain an exact distance (see the SDF discussion next).

12.7 Many Functions, One Surface (slide 53)

The surface does not fix f : many different fields have the same zero set. So one can choose an f that is useful *away* from the surface.

12.8 Signed Distance Function (slides 54–58)

Definition 12.5 (Signed distance function (SDF)).

$$f(\mathbf{x}) = \begin{cases} -\text{dist}(\mathbf{x}, \mathcal{S}) & \mathbf{x} \text{ inside} \\ +\text{dist}(\mathbf{x}, \mathcal{S}) & \mathbf{x} \text{ outside} \end{cases}, \quad \text{dist}(\mathbf{x}, \mathcal{S}) = \min_{\mathbf{y} \in \mathcal{S}} \|\mathbf{x} - \mathbf{y}\|.$$

Which of the fields of slide 55 are SDFs? $x^2 + y^2 + z^2 - 1$ and the quartic of slide 49: *neither*!

Proposition 12.6 (Eikonal condition). *An SDF satisfies $\|\nabla f\| = 1$ (wherever it is differentiable).*

Proof. Take a small step of length δ from a point. In *any* direction the distance to the surface changes by at most δ ; stepping straight away from the closest surface point it changes by exactly δ . So the steepest slope of f is exactly one. $\square$

Example 12.7. $f = \|\mathbf{x}\| - 1$: $\nabla f = \mathbf{x}/\|\mathbf{x}\|$, $\|\nabla f\| = 1$, an SDF. $f = \|\mathbf{x}\|^2 - 1$: $\nabla f = 2\mathbf{x}$, $\|\nabla f\| = 2\|\mathbf{x}\| \neq 1$, not an SDF.

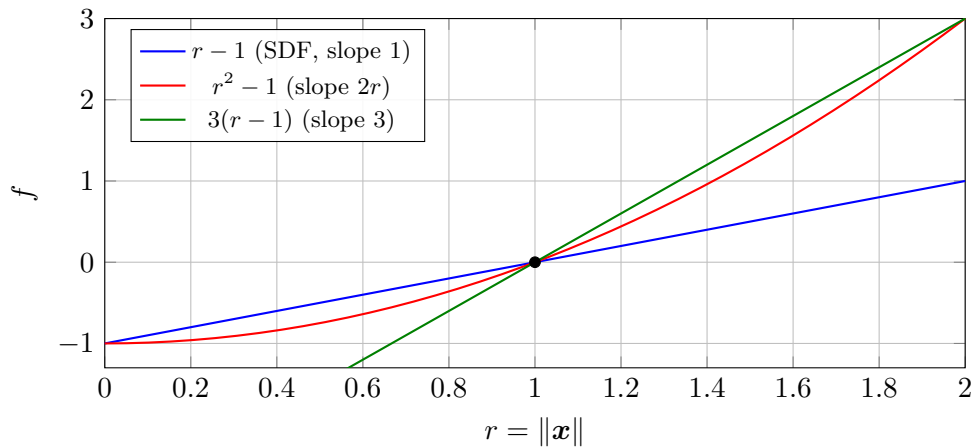


Figure 30: Three fields with the same zero set (the unit sphere, $r = 1$). Only the slope-one profile is a signed distance function.

Intuition

The eikonal condition says “the field grows like a ruler as you walk away from the surface”. An SDF is the most *honest* field: its value literally is the distance to the surface. The other fields agree on *where* the surface is but exaggerate or shrink the distances; $r^2 - 1$ overstates by the factor $r + 1$ at radius r .

Why honesty matters: safe steps (*slides 57–58*).

Proposition 12.8 (Safe step). *If f is an SDF, the ball of radius $|f(\mathbf{x})|$ around $\mathbf{x}$ contains no surface point in its interior. Hence a ray can safely advance by $|f(\mathbf{x})|$. If f overstates the distance, the ball can cross the surface and stepping by f can jump through it.*

Proof. By definition $|f(\mathbf{x})| = \text{dist}(\mathbf{x}, \mathcal{S})$, the radius of the largest ball around $\mathbf{x}$ that avoids $\mathcal{S}$. For a non-SDF field the ball of radius $|f|$ can be larger than $\text{dist}(\mathbf{x}, \mathcal{S})$ and then contains surface points. $\square$

Example 12.9 (Slides 57–58, $\mathbf{x}$ at $r \approx 2.67$). For $f = r^2 - 1$: $f = 6.14$, true distance 1.67, $\|\nabla f\| = 5.34$: the circle of radius $|f|$ crosses the surface. For an SDF at the same point the ball of radius $r - 1 = 1.67$ touches the surface without crossing.

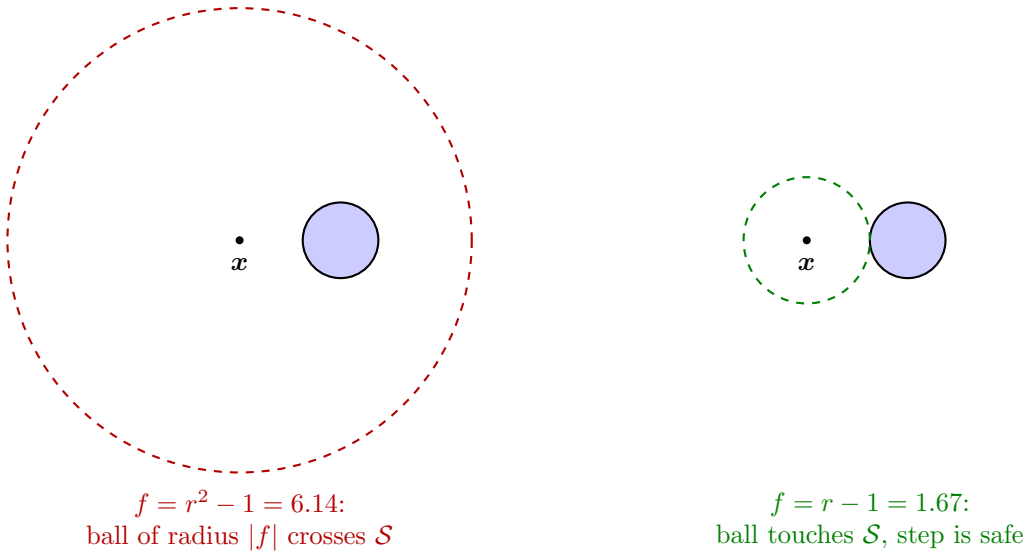


Figure 31: The same point $\mathbf{x}$ with two fields that define the same sphere. Left: stepping by $f = r^2 - 1$ overshoots. Right: an SDF gives the largest safe step.

Algorithm 12.10 (Sphere tracing / ray marching an SDF). Given a ray $\mathbf{o} + t\mathbf{d}$ ($\|\mathbf{d}\| = 1$) and an SDF f : set $t \leftarrow 0$; repeat $t \leftarrow t + f(\mathbf{o} + t\mathbf{d})$ until $f < \varepsilon$ (hit) or t exceeds a maximum (miss).

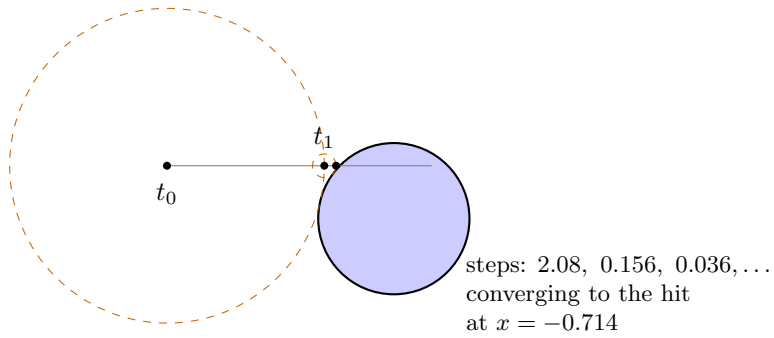


Figure 32: Sphere tracing a horizontal ray at height 0.7 against the unit sphere: each step is the SDF value, the radius of the empty ball.

Intuition

It is like walking through fog toward a wall with a distance meter: if the meter never lies (SDF) you can always walk the displayed distance without hitting the wall, and the steps shrink as you approach. A meter that overstates makes you walk into and through the wall. This is why the table on slide 66 lists “ray marching” as the way to render implicit surfaces.

12.9 Storing a Field on a Grid (*slide 59*)

Store samples $F[i, j, k] = f(\mathbf{x}_{ijk})$ at the grid points and interpolate in between: the same data structure as a voxel grid, but holding *signed distances* rather than occupancy. A *TSDF* keeps values only near the surface and clamps them to $\pm\tau$ elsewhere, $f_\tau = \text{clamp}(f, -\tau, \tau)$; it is the standard way to fuse many depth maps into one model, as in KinectFusion.

```
xs = np.linspace(-1.5, 1.5, 128)
X, Y, Z = np.meshgrid(xs, xs, xs, indexing="ij")
F = np.sqrt(X**2 + Y**2 + Z**2) - 1.0 # unit-sphere SDF on a 128^3 grid
```

Intuition

The grid now stores *how far* each cell is from the surface instead of *whether* it is occupied. Values change smoothly across the surface, so interpolation places the surface at sub-voxel accuracy, and the clamped version stores only what matters: a thin shell around the surface.

12.10 Storing a Field in a Network (*slide 60*)

$\{\mathbf{x} \mid f_\theta(\mathbf{x}) = 0\}$, with f_θ a small MLP (three numbers in, one out). Two requirements: $f_\theta(\mathbf{x}_i) \approx 0$ on the data points (the surface passes through the data), and $\|\nabla f_\theta(\mathbf{x})\| \approx 1$ everywhere (it behaves like an SDF). Resolution is not tied to a grid.

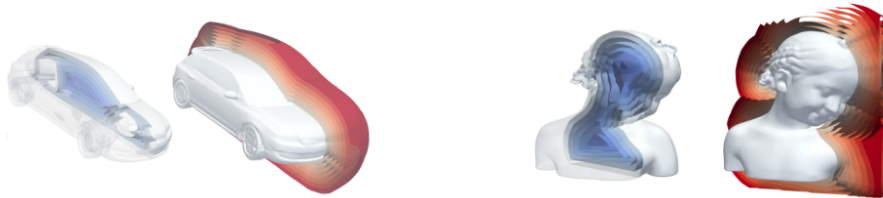


Figure 33: A small MLP can represent a continuous field f_θ , whose zero set is the surface.

Example 12.11 (One way to write the two requirements as a loss). $\mathcal{L}(\theta) = \frac{1}{N} \sum_i |f_\theta(\mathbf{x}_i)| + \lambda \mathbb{E}_{\mathbf{x}} [(\|\nabla f_\theta(\mathbf{x})\| - 1)^2]$. The first term pulls the zero set through the samples; the second is the eikonal penalty of Proposition 12.6.

Intuition

The network is a *compressed*, resolution-free grid: query it at any point, at any density. The eikonal penalty is what stops the network from producing an arbitrary function that merely vanishes on the data.

12.11 From Implicit Surface to Mesh (slide 61)

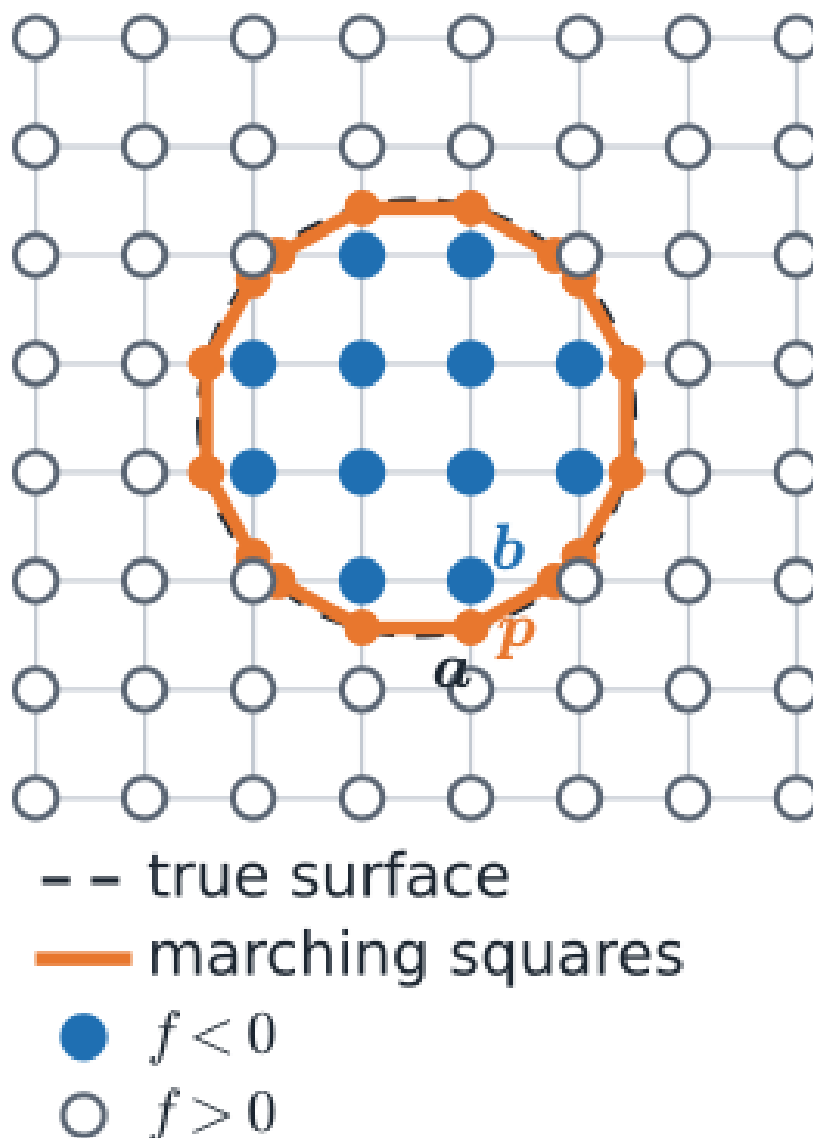


Figure 34: Marching squares (the 2D version of marching cubes) in one cell. The blue corners are inside ($f < 0$), the white corners are outside ($f > 0$). The dotted line is the true surface; the orange line is the interpolated surface from marching squares.

Algorithm 12.12 (Marching cubes). **1.** Evaluate f on a grid (analytic, a TSDF, or a network).

2. Find the cells whose corners differ in sign.

3. On each such edge $[a, b]$, place a vertex at the zero crossing $p = a + t(b - a)$ with $t = \frac{f(a)}{f(a) - f(b)}$.

4. In 3D, join the vertices with a lookup table on the corner signs: 2^8 cases per cube.

```
from skimage.measure import marching_cubes
```

```

h = xs[1] - xs[0] # voxel size, from the grid above
verts, faces, normals, _ = marching_cubes(F, level=0.0, spacing=(h, h, h))
verts += xs[0] # grid -> world coordinates

```

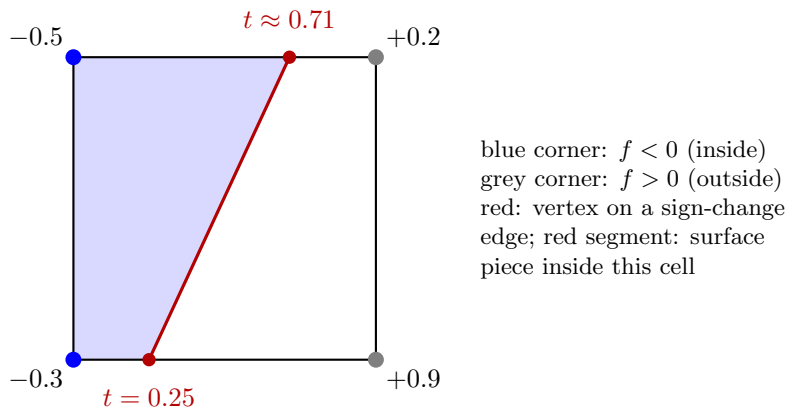


Figure 35: Marching squares (the 2D version of marching cubes) in one cell. On the bottom edge $t = \frac{-0.3}{-0.3-0.9} = 0.25$; on the top edge $t = \frac{-0.5}{-0.5-0.2} \approx 0.71$. The left and right edges have no sign change, hence no vertex.

Intuition

Marching cubes is “connect the dots along the shoreline”: the corner signs say which cell edges the coastline crosses; linear interpolation of f along that edge says *where* (a corner with a small $|f|$ is close to the surface, so the vertex sits close to it). This is the bridge from implicit to mesh, which is what GPUs render.

12.12 Implicit Surfaces: Summary (*slide 62*)

Easy	Hard
inside or outside: the sign of f	getting points on the surface
any topology, changing continuously	no direct rasterization
booleans: a min or a max	an arbitrary f may define nothing
normals: ∇f	

The mirror image of parametric surfaces.

13 Boundary vs Volumetric Representations (*slide 63*)

Slide 63 summarizes the two families: *boundary* representations for shapes (point clouds, meshes, parametric surfaces) and *volumetric* representations for shapes (voxels, implicit fields).

14 Density Fields (*slides 64–65*)

14.1 Density fields (*slide 64*)

Occupancy asks “is this point inside?”; density asks “how opaque is this point?”.

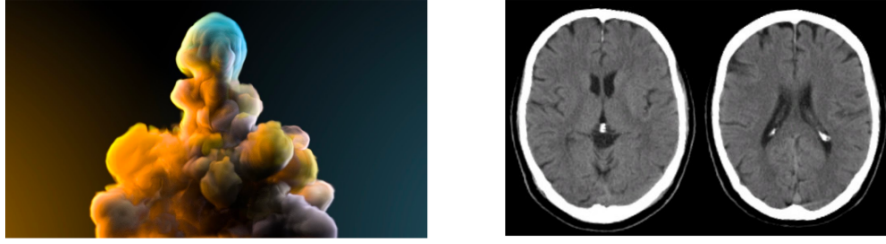


Figure 36: Two images examples

Definition 14.1 (Density field). A density field assigns $\sigma(\mathbf{x}) \in \mathbb{R}^+$ (or $\sigma[i, j, k] \in \mathbb{R}^+$ on a grid); $\sigma(\mathbf{x}) \geq 0$ says how much light is absorbed per unit length at $\mathbf{x}$.

There is no notion of a surface: e.g. $\sigma(\mathbf{x}) = 0.1$ everywhere represents a semi-transparent fluid. Typical examples on the slide: smoke and CT-scan slices. The lecture will revisit this in more detail later (NeRF).

Example 14.2 (Preview: how much light survives?). If a beam crosses a slab of length L of constant density σ , the fraction of light transmitted is $e^{-\sigma L}$ (absorption per unit length compounds multiplicatively). For $\sigma = 0.1$ and $L = 10$: $e^{-1} \approx 0.37$. (This is a standard consequence of the “absorbed per unit length” reading; the slides develop it later.)

14.2 Primitive-based Density Fields (*slide 65*)

$$\sigma(\mathbf{x}) = \sum_k w_k \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^\top \Sigma_k^{-1}(\mathbf{x} - \boldsymbol{\mu}_k)\right).$$

These are *hybrid* representations for primitive-induced density fields (blobs/Gaussians with weights w_k , centres $\boldsymbol{\mu}_k$, covariances Σ_k); the lecture will revisit them in more detail (differentiable Gaussian rendering).



Figure 37: Images examples of primitive-based density fields

Intuition

Instead of a grid of numbers, the density is a *cloud of soft blobs*: each blob has a position, a size/orientation (via Σ_k) and a weight. The whole field can be edited by moving blobs, and only regions with blobs cost anything, so it avoids the “pay for empty space” problem of dense grids.

15 So? (*slide 66*)

Slide 66 compares the representations side by side (its cells are colour-coded easy / possible with effort / hard; the colours do not survive in the text of the PDF, so the qualitative ratings appear in Section 17, based on the earlier slides):

	Points on the surface	Inside or outside?	Continuous edits reach	Rendering	Typical use
Point cloud (array $N \times 3$)	given	no surface	any point set	splats	sensors: LiDAR, RGB-D
Mesh ($V : N \times 3$, $F : M \times 3$)	sample triangles	count ray crossings	same wiring only	GPU rasterizer	graphics, games
Parametric ($f(\mathbf{u})$)	evaluate f	search over $\mathbf{u}$	same topology only	via a mesh	CAD, design
Voxels (grid $W \times H \times D$)	marching cubes	look up the cell	any topology*	blocky	3D CNNs, occupancy maps
Implicit (SDF) ($f(\mathbf{x})$: formula, grid, net)	marching cubes	sign of f	any topology	ray marching	reconstruction, neural fields

* with occupancy probabilities, not 0 or 1.

Intuition

No row wins every column: pick the representation for the task. Need to shoot rays fast on a GPU? Mesh. Need to change topology or run inside/outside queries? Implicit. Data straight from a LiDAR? Points. Feeding a 3D CNN? Voxels.

16 And many more... (slides 67–71)

16.1 Octrees (slide 68)

Octrees are *voxels with adaptive resolution*: surface-like efficiency for rendering and storage, volume-like efficiency for occupancy queries; but it is difficult to edit or predict the sparsity structure.

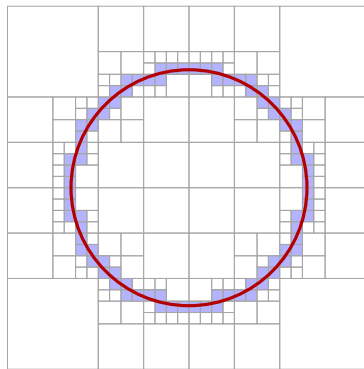


Figure 38: A quadtree (2D octree) for a disk of radius 0.65 in $[-1, 1]^2$, subdivided down to 5 levels only where the boundary passes (blue). It uses 244 leaf cells; a uniform grid of the same finest resolution (32×32) uses 1024.

Intuition

An octree spends resolution where the shape is: big cells for air and interior, tiny cells on the boundary. The trade-off is structural: the tree *shape* depends on the data, which is hard to edit or to predict with a network (the same sparse/dense tension seen with meshes' F).

16.2 Layered Depth Images (*slide 69*)

Definition 16.1 (Layered depth image (LDI)). Layer 1 is the visible content (depth and colour per pixel); layer $N+1$ stores the depth and colour of the *next* surface point the ray hits after layer N . This extends depth to capture “full” 3D, but the layers are often very discontinuous.

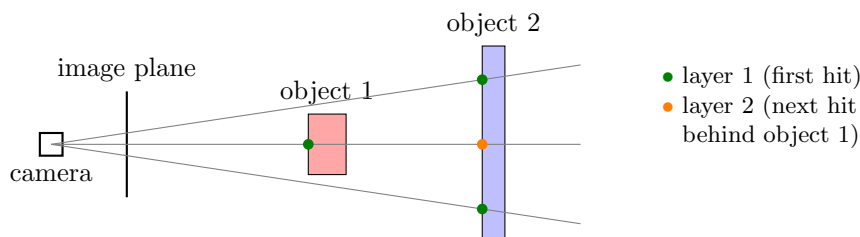


Figure 39: Three pixel rays. The upper and lower rays hit only object 2 (layer 1 only); the middle ray hits object 1 first and object 2 second, so its pixel stores two (depth, colour) entries.

Intuition

A depth map keeps only the first hit along each ray; an LDI keeps a *list* of hits. The hidden second layer (what is behind object 1) is what allows rendering from a slightly different viewpoint without holes; but the layer index jumps around from pixel to pixel, so layers are not smooth images.

16.3 Multi-plane Images (*slide 70*)

Multi-plane images (MPIs) are discrete volumetric representations with *non-uniform* spatial cells: a stack of fronto-parallel planes at fixed depths, each a (colour, transparency) image. They are extremely efficient to render (each layer is just a plane at fixed depth), but have limited resolution for far-away content; uniform cells may be preferable for object-centric 3D.

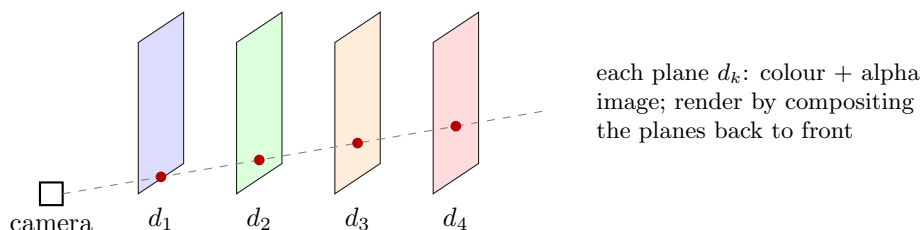


Figure 40: A multi-plane image: planes at increasing depth $d_1 < \dots < d_4$; a pixel ray crosses all of them and the result is their composite.

Intuition

An MPI is a pile of transparent slides at fixed depths: to draw the scene from a nearby viewpoint, shift and blend the slides. Planes are spaced non-uniformly (dense near, sparse far), which is why distant content gets little resolution.

16.4 CAD Models and CSG Trees (*slide 71*)

CAD models and CSG (constructive solid geometry) trees are *compositions of primitive volumes or surfaces*. They are popular across design applications but challenging for learning.

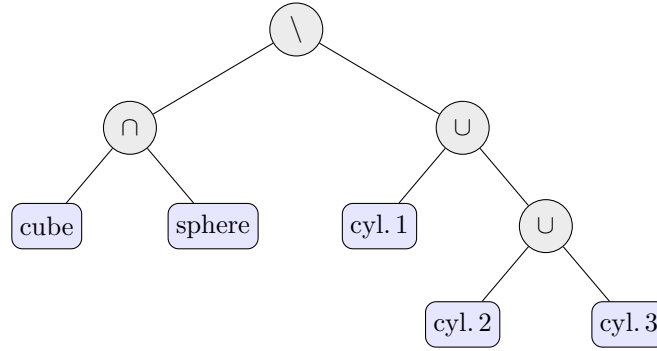


Figure 41: A CSG tree: leaves are primitives, inner nodes are Boolean operations (as on slide 71: $(\text{cube} \cap \text{sphere}) \setminus (\text{cyl.}_1 \cup (\text{cyl.}_2 \cup \text{cyl.}_3))$).

Example 16.2 (A CSG tree is a nested min / max). If each primitive has an implicit function ($f_{\text{cube}}, f_{\text{sph}}, f_{c1}, f_{c2}, f_{c3}$), the tree of the figure is the field

$$f = \max \left(\max(f_{\text{cube}}, f_{\text{sph}}), -\min(f_{c1}, \min(f_{c2}, f_{c3})) \right),$$

by the Boolean rules of slide 52.

Intuition

A CSG model is a *program*: a recipe of primitives and Boolean steps. It is compact, editable and exact, but the structure (which tree, how deep) is discrete and combinatorial, so it is hard to predict with a neural network by gradient descent.

17 Summary (*slides 72–74*)

The summary slide (72) repeats the zoo of slide 4: mesh, point cloud, CAD model, voxels, implicit surface $\{\mathbf{x} \mid f(\mathbf{x}) = 0\}$, parametric surface $f(\mathbf{u}) = \mathbf{x} \in \mathbb{R}^3$, CSG tree, and many more. Slides 73–74 are “Questions?” and Acknowledgments: the slides are adapted from *Learning for 3D Vision*, CMU 16-825, Prof. Shubham Tulsiani.

Comparison table

	Get surface points	Inside / outside	Change topology continuously	Render directly	Memory scales with
Point cloud	given	no surface	any point set	splats	points (N)
Triangle mesh	sample triangles	count ray crossings	same wiring only	rasterizer	vertices, faces
Parametric	evaluate f	search over $\mathbf{u}$	same topology only	via a mesh	parameters of f
Voxels	marching cubes	look up cell	with probabilities	blocky	n^3
Implicit / SDF	marching cubes	sign of f	any topology	ray marching	grid n^3 or network

easy possible with effort hard / limited . Ratings are our reading of slides 22–62 and the table of slide 66, not a table from the slides.

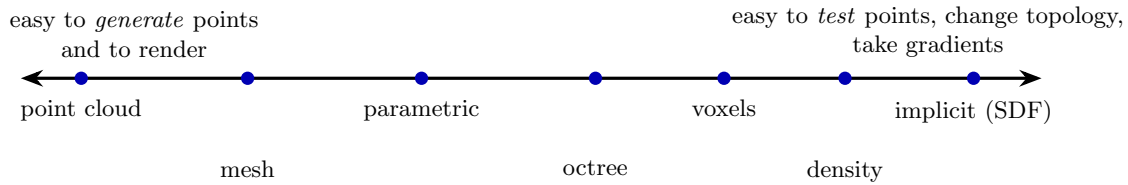


Figure 42: A qualitative map of the “generate vs. test” spectrum (our summary of slides 41, 48, 62 and 66; positions are indicative only).

Key take-aways

- **There is no universal 3D representation.** Points and depth maps come from sensors; meshes are for rendering and graphics; implicit fields are for inside/outside tests, topology changes and learning; the right choice depends on the task.
- **Parametric generates, implicit tests.** $f(\mathbf{u}) = \mathbf{x}$ makes sampling trivial and queries hard; $\{f(\mathbf{x}) = 0\}$ makes queries trivial (sign of f , ∇f for normals, min / max for Booleans) and sampling hard.
- **Meshes split geometry from connectivity.** V is continuous and optimizable; F is discrete, so predicting arbitrary connectivity or changing topology is hard; check winding order, and remember $F \approx 2V$ for closed genus-0 meshes.
- **Grids are simple but pay for empty space.** A n^3 voxel grid (or SDF grid) handles any topology and suits convolutions, but only $\sim n^2$ cells touch the surface; octrees and continuous fields (networks, Gaussians) fight this cost.
- **SDFs are the well-behaved implicit fields.** $\|\nabla f\| = 1$ makes $|f|$ the true distance, enabling safe ray-marching steps and TSDF fusion, and marching cubes converts any such field back into a mesh.